CoE 163 Computing Algorithms and Architecture

Adrian Campo - 2022-00945
John Jovic Gutierrez - 2022-07198

Problem Statement:

Combination locks are one of the easiest locks to pick without specialized equipment. These locks are unlocked when the dials are set to the unlock combination. Each dial contains digits from 0 to 9, and the numbers wrap around (i.e. incrementing from 9 will turn that dial to 0). Anyone can "brute-force" unlock them by trying out each and every combination possible. You are working for a lock company and you are assigned to determine the minimum number of turns needed to unlock a combination lock given its initial and unlocked final value for quality control purposes. A "turn" is defined as an increment or decrement in value of a digit by one. For example, if the initial configuration of a lock shows 1239 and the unlock value is 2348, only four (4) turns are needed, like so

1 -> 2 : 1 turn
2 -> 3 : 1 turn
3 -> 4 : 1 turn
9 -> 8 : 1 turn

Since you expect to compute locks with an arbitrary number of combination dials, you decided to write a program that will compute the minimum number of turns needed to unlock a combination lock of N locks, given a starting point value.

I. Algorithm

Initially, we thought of checking the relative distance of each digit as we traverse the digits on the said combination lock. We calculated the distance say delta(x) by solving the absolute difference of the final and initial position respectively. This delta(x) represents the positive distance between the two numbers in our number line ranging from [0,9] inclusive. Noting that the maximum (minimum of the distance between two points) is five. Therefore we realized that we were faced with an issue that this delta(x) is not yet the minimum distance of the two points in our number line and also realize that we have to consider the fact that we can count one turn from end points to end points (i.e. from 9 to 0 and vice versa). Thus, we thought of halving the number line to basically have four end points, [0,4] and [5,9] inclusive. From here, we tried to apply our delta(x) but then we have come to realize that this method not only is computationally exhaustive but also includes case to case basis, which means we have to use if statements in our program.

Eventually, we thought of changing our perspective with counting in the number line. Since we are looking for the minimum distance between two points, we thought of a wrap around for the problem of "counting back". In this case, we are given two points, and with these two points, we want to offset the smaller number (point) by 10 relative to its original position in the number line. And then apply the delta(x) function between the largest number(point) and the "offsetted" number. The result of this is then compared to the normal case wherein the delta(x) function is applied to the original numbers (points).

Building on these ideas, what if the mix of two. Instead of having two comparison (two conditional statements), what if we skip the offsetting part. Now, we are going to be more focused on utilizing the delta(x) function. And the idea of offsetting was our starting point of using the number 10 which is the total number of points in our number line. Having this in mind, we used this to with the result of delta(x) function. In this case, we are only using one conditional statement to find the minimum distance between two numbers within our number line. Thus, we were able to find the minimum distance by comparing the results of delta(x) function and 10 - delta(x).

Here is the code snippet of the algorithm described:

```
n_values = range(10, 101, 10)
iterations = 1000
   ...
for n in n_values:
   ...
    for _ in range(iterations):
        t0 = time.perf_counter()
        total_turns = 0
         for i in range(n):
              s_digit = int(start_val[i])
              e_digit = int(end_val[i])
              diff = abs(s_digit - e_digit)
              total_turns += min(diff, 10 - diff)
              print(total_turns)
        t1 = time.perf_counter()
        runtimes.append(t1 - t0)
   ...
```

II. Analysis of Algorithm

A. Asymptotic Analysis

The algorithm easily processes the the individual operations with a Time Complexity of O(1), however, since we are iterating through a loop with size n, sizing the worst case of the time complexity to **O(n)**. Since we simply used the array data structure to iterate through the varying length of locks (n), we linearly set this as the space complexity of the algorithm as **O(n)**. We thought of optimizing the space complexity by utilizing module and division operations to iterate through the padlock theoretically optimizing it into a space complexity of **O(1)**, however implementing the mathematical optimization is computationally exhaustive and we are more comfortable with using arrays, and concern with the complexity of implementation.

```
   n_values = range(10, 101, 10)
  iterations = 1000
...
  for n in n_values: # n are randomly generated
   ...
      for _ in range(iterations):
          t0 = time.perf_counter()
          total_turns = 0
       for i in range(n): #O(n)
           s_digit = int(start_val[i]) # O(1)
           e_digit = int(end_val[i]) # O(1)
           diff = abs(s_digit - e_digit) # O(1)
           total_turns += min(diff, 10 - diff) # O(1)
           print(total_turns) # O(1)
          t1 = time.perf_counter()
          runtimes.append(t1 - t0)
...
```

III. Profiling

What takes up time in our algorithm is that for each dial, the main time cost is the Python-level overhead. Additionally, operations such as arithmetic, min selection using the min(_) function, the loop overhead, the indexing into strings, and the digit conversion using the int(_) function which is relatively expensive in Python because it parses a character and construct an integer object. These are due to the fact that we opted to use array/string data structure instead of more complex mathematical modules, these type-casting and arithmetic operations run exactly N times per calculations.

As established in the problem statement, the lock length dictates the total number of loops the algorithm must perform. So, the space complexity is also linearly tied to this length at O(n). Simply put a longer lock means a proportionally longer runtime. Moreover, the concept of memory cache plays a crucial role in the runtime variances as we initially hypothesized. We deemed it useful to discuss how the starting position of the lock affects the runtimes. Due to our approach of sequential data structure, string and arrays are stored contiguously in the main memory. With that in mind, the very first iteration, the CPU accesses the starting position of the lock, it loads it into its cache (which inquires a cold-start cache miss). Noting that in this case, the starting lock position changes in every iteration. Thus, in the succeeding iterations, if the data is already in the cache (a cache hit), its reads are extremely fast leading to the low “Min Time” as observed. If the algorithm must fetch a new string from main memory (a cache miss) due to iterating through a new generated combination, the CPU stalls. This main memory access latency is expensive and is likely what causes the “Max Time” as observed.

A. Effect of lock length on the runtimes:

```python
for n in n_values:
    start_val, end_val = generate_test_case(n)
    runtimes = []
    t0 = time.perf_counter()
    # Requirement: Loop at least 1000 times
    for _ in range(iterations):
        find_min_combi(n, start_val, end_val)
        t1 = time.perf_counter()
        runtimes.append(t1 - t0)

    avg_run = sum(runtimes) / iterations
    max_run = max(runtimes)
    min_run = min(runtimes)
```

N	Avg Time (s)	Min Time (s)	Max Time (s)
10	0.00230410	0.00001540	0.00431310
20	0.00351831	0.00001160	0.00687580
30	0.00540204	0.00001430	0.01194230
40	0.00917373	0.00001580	0.01904890
50	0.00978826	0.00001970	0.02198420
60	0.01409072	0.00002220	0.02885490
70	0.01739782	0.00003410	0.03439830
80	0.02089884	0.00003940	0.04105780
90	0.01671019	0.00003440	0.03589070
100	0.02022403	0.00003780	0.04503760

B. Effect of the starting position of the lock on the runtimes:

```python
def generate_test_case(n):
    start = "".join([str(random.randint(0, 9)) for _ in range(n)])
    return start
```

```python
for n in n_values:
    end_val = generate_test_case(n)
    runtimes = []
    t0 = time.perf_counter()
    # Requirement: Loop at least 1000 times
    for _ in range(iterations):
        start_val = generate_test_case(n)
        find_min_combi(n, start_val, end_val)
        t1 = time.perf_counter()
        runtimes.append(t1 - t0)

    avg_run = sum(runtimes) / iterations
    max_run = max(runtimes)
    min_run = min(runtimes)
```

N	Avg Time (s)	Min Time (s)	Max Time (s)
10	0.00807681	0.00002100	0.01701220
20	0.01525091	0.00003660	0.03040070
30	0.02089784	0.00003440	0.04227320
40	0.02347243	0.00004570	0.05360240
50	0.04320138	0.00008800	0.08546810
60	0.04682941	0.00011510	0.09213770
70	0.05364706	0.00012890	0.10525730
80	0.03839181	0.00007840	0.07543670
90	0.05479768	0.00008730	0.11578640
100	0.04960160	0.00010100	0.11118290

N	Avg Time (s)	Min Time (s)	Max Time (s)
10	0.00000357	0.00000328	0.00000853
20	0.00000765	0.00000648	0.00006352
30	0.00001025	0.00000958	0.00002858
40	0.00001479	0.00001269	0.00013548
50	0.00001735	0.00001550	0.00015066
60	0.00002255	0.00001897	0.00025745
70	0.00002383	0.00002202	0.00010671
80	0.00005204	0.00002504	0.00032766
90	0.00007829	0.00004134	0.00011765
100	0.00007866	0.00004425	0.00013722

N	Avg Time (s)	Min Time (s)	Max Time (s)
10	0.00000398	0.00000331	0.00003245
20	0.00000684	0.00000640	0.00002032
30	0.00001170	0.00000959	0.00016715
40	0.00001629	0.00001266	0.00021632
50	0.00001752	0.00001567	0.00014978
60	0.00002550	0.00001896	0.00006065
70	0.00002602	0.00002186	0.00024101
80	0.00003011	0.00002505	0.00019430
90	0.00003534	0.00002789	0.00021770
100	0.00003544	0.00003096	0.00033460

N	Avg Time (s)	Min Time (s)	Max Time (s)
10	0.00000364	0.00000331	0.00001088
20	0.00001125	0.00000631	0.00012955
30	0.00001292	0.00000941	0.00014163
40	0.00001566	0.00001262	0.00012484
50	0.00002492	0.00001568	0.00027835
60	0.00002923	0.00001900	0.00032751
70	0.00002955	0.00002184	0.00016716
80	0.00003375	0.00002511	0.00016676
90	0.00006197	0.00002857	0.00020057
100	0.00004149	0.00003131	0.00020764

IV. Assembly

Below is our own attempt for assembly, it is a simple attempt that follows our attempt in python logic. In an attempt to convert to Godbolt's compilers, we rewrite the python code into C++, see Appendix B, we also tried to compile it with varying levels of optimization. We have observed that higher levels of optimization in our algorithm in bare assembly is the equivalent of having to assembly code in registers; it made it so that it has implemented in manual operation of memory location. At maximum optimization option, `-O3` , It iterate the operation at the memory locations at long lines of code.

A. Own Attempt for Assembly

```
find_min_turns:
    xor rax, rax
    xor rcx, rcx

loop_start:
    cmp rcx, rdx
    jge loop_end

    ; Load digits
    mov r8, byte [rdi + rcx]
    mov r9, byte [rsi + rcx]

    ; Calculate diff = start_digit - end_digit
    mov r10, r8
    sub r10, r9             ; r10 = r8 - r9

    ; Absolute value of diff
    cmp r10, 0
    jge is_positive
```

```
    neg r10

is_positive:
    ; At this point, r10 = |s - e|
    ; Calculate wrap_around = 10 - r10
    mov r11, 10
    sub r11, r10

    ; Find minimum of r10 and r11
    cmp r10, r11
    jle take_direct
    mov r10, r11
take_direct:

    ; Accumulate result
    add rax, r10

    inc rcx
    jmp loop_start

loop_end:
    ret
```

B. At Minimal Optimization -01

```
findMinCombi(int, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocat
        test    edi, edi
        jle     .L4
        mov     r10, QWORD PTR [rsi]
        mov     r9, QWORD PTR [rdx]
        movsx   rdi, edi
        mov     ecx, 0
        mov     esi, 0
        mov     r8d, 10
.L3:
        movsx   edx, BYTE PTR [r10+rcx]
        movsx   eax, BYTE PTR [r9+rcx]
        sub     edx, eax
        mov     eax, edx
        neg     eax
        cmovs   eax, edx
        mov     edx, r8d
        sub     edx, eax
        cmp     edx, eax
        cmovg   edx, eax
        add     esi, edx
        add     rcx, 1
        cmp     rcx, rdi
        jne     .L3
.L1:
        mov     eax, esi
        ret
.L4:
        mov     esi, 0
        jmp     .L1
```

C. At Optimization Level -02

```
findMinCombi(int, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocat
        mov     r9d, edi
        test    edi, edi
        jle     .L29
        lea     eax, [r9-1]
        mov     r8, QWORD PTR [rsi]
        mov     rdi, QWORD PTR [rdx]
        cmp     eax, 14
        jbe     .L30
        mov     edx, r9d
        mov     esi, 655370
        pxor    xmm6, xmm6
        xor     eax, eax
        shr     edx, 4
        movd    xmm3, esi
        sal     rdx, 4
        pshufd  xmm3, xmm3, 0
```

D. At Maximum Optimization -03

```
findMinCombi(int, std::__cxx11::basic_string<char, std::char_traits<char>, std::allocat
        test    edi, edi
        jle     .L17
        lea     eax, [rdi-1]
        mov     rcx, QWORD PTR [rsi]
        mov     rdx, QWORD PTR [rdx]
        cmp     eax, 14
        jbe     .L18
        mov     esi, edi
        mov     r11d, 655370
        pxor    xmm6, xmm6
        xor     eax, eax
        shr     esi, 4
        movd    xmm3, r11d
        sal     rsi, 4
        pshufd  xmm3, xmm3, 0
.L14:
        movdqu  xmm0, XMMWORD PTR [rcx+rax]
        movdqu  xmm1, XMMWORD PTR [rdx+rax]
        pxor    xmm9, xmm9
        pxor    xmm8, xmm8
        add     rax, 16
        pcmpgtb xmm9, xmm0
        pcmpgtb xmm8, xmm1
        movdqa  xmm2, xmm0
        movdqa  xmm7, xmm1
        punpcklbw       xmm7, xmm8
        punpcklbw       xmm2, xmm9
        punpckhbw       xmm1, xmm8
        psubw   xmm2, xmm7
        pxor    xmm7, xmm7
        punpckhbw       xmm0, xmm9
        psubw   xmm7, xmm2
```

V. Appendix

A. The Whole Program in Python

```python
import time
import random

def find_min_combi(num_dials, start_str, end_str):
    # Calculates minimum turns.
    # Using strings directly to handle N up to 100 easily.
    total_turns = 0
    for i in range(num_dials):
        s_digit = int(start_str[i])
        e_digit = int(end_str[i])
        diff = abs(s_digit - e_digit)
        total_turns += min(diff, 10 - diff)
    return total_turns

def generate_test_case(n):
    #Generates two random strings of length n.
    start = "".join([str(random.randint(0, 9)) for _ in range(n)])
    end = "".join([str(random.randint(0, 9)) for _ in range(n)])
    return start, end

if __name__ == "__main__":
    # Test cases for N = 10, 20, 30, ..., 100
    n_values = range(10, 101, 10)
    iterations = 1000

    print(f"{'N':<5} | {'Avg Time (s)':<15} | {'Min Time (s)':<15} | {'Max Time (s)':<15}")
    print("-" * 60)

    for n in n_values:
        start_val, end_val = generate_test_case(n)
        runtimes = []

        # Requirement: Loop at least 1000 times
        t0 = time.perf_counter()
        for _ in range(iterations):
            find_min_combi(n, start_val, end_val)
            t1 = time.perf_counter()
            runtimes.append(t1 - t0)

        avg_run = sum(runtimes) / iterations
        max_run = max(runtimes)
        min_run = min(runtimes)

        print(f"{n:<5} | {avg_run:<15.8f} | {min_run:<15.8f} | {max_run:<15.8f}")
```

B. Whole Code Rewritten in C++

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <chrono>
#include <cmath>
#include <algorithm>

using namespace std;
using namespace std::chrono;

int findMinCombi(int numDials, string start, string end) {
    int totalTurns = 0;

    for (int i = 0; i < numDials; ++i) {
        int sDigit = start[i] - '0';
        int eDigit = end[i] - '0';
        int diff = abs(sDigit - eDigit);
        totalTurns += min(diff, 10 - diff);
    }
    return totalTurns;
}

int main() {
    int n = 4;
    string start = "1239";
    string end = "2348";

    int iterations = 1000;
    long long totalDuration = 0;

    auto t1 = high_resolution_clock::now();
    for (int i = 0; i < iterations; ++i) {
        findMinCombi(n, start, end);
    }
    auto t2 = high_resolution_clock::now();

    auto duration = duration_cast<nanoseconds>(t2 - t1).count();

    cout << "Minimum turns: " << findMinCombi(n, start, end) << endl;
    cout << "Total time for " << iterations << " runs: " << duration << " ns" << endl;
    cout << "Average time per run: " << duration / iterations << " ns" << endl;

    return 0;
}
```